

Are We Still Programming, or Just Prompting the Machine?

Diana Avila

CSc 4810: Artificial Intelligence

Department of Computer Science

Georgia State University

Email: davila2@student.gsu.edu

Abstract—Recent advances in AI-assisted coding have created a practical question for computer science students: if machines can now generate substantial amounts of code, what should future developers learn? This paper examines that question through a structured synthesis of recent developer surveys, labor-market studies, workflow research, and software-quality evidence. The paper first reviews current methods of AI-assisted programming, including code completion, conversational code generation, and so-called vibe coding. It then proposes a new analytical method, the Hybrid Responsibility Framework, which separates AI-assisted software work into three layers: generation, verification, and accountability. The framework is implemented by mapping recent evidence to each layer and using that mapping to evaluate what has changed in programming practice and what has not. The analysis shows that AI tools are now mainstream for drafting code and searching for information, but trust remains limited, performance on complex tasks is uneven, and responsibility-heavy work such as debugging, security, architecture, deployment, and maintenance still depends strongly on human expertise. Labor-market evidence also suggests a mixed picture: broad software demand remains positive, while early-career workers may face a narrower entry path. The main conclusion is that programming is not being replaced by prompting. Instead, software work is being reorganized around higher-level specification, verification, and judgment. For students, the most robust strategy is to develop both deep coding fundamentals and strong prompt-based specification skills.

Index Terms—AI-assisted coding, programming education, vibe coding, software engineering, labor market, developer tools

I. INTRODUCTION

One of the most striking changes in software development is that code increasingly begins as a conversation. A student can describe a feature in plain language, receive a working draft in seconds, and watch a tool generate functions, tests, and helper files that previously required careful manual effort. This new workflow has made software creation feel faster and more accessible, but it has also created uncertainty about what counts as real programming skill. If a system can generate code, explain APIs, and suggest fixes, what should students still learn? That question is now central to computer science education because it affects coursework, portfolio building, internship preparation, and long-term career planning.

Current public discussion often reduces the issue to a false choice: either learn to code or learn to prompt. That framing is too narrow. Prompting is not simply a replacement

for programming syntax. In practice, it acts as a higher-level interface for specifying goals, constraints, and desired outcomes. At the same time, the output still has to be reviewed, tested, debugged, secured, and maintained. The real question, then, is not whether programming disappears, but which parts of programming are changing and which parts remain durable.

This paper addresses that problem through an evidence-based synthesis rather than through hype or anecdote. The contributions are threefold. First, it summarizes current methods and findings from recent publications on AI-assisted coding, developer trust, labor-market effects, and code quality. Second, it proposes a new analytical method for interpreting these changes: the Hybrid Responsibility Framework. Third, it applies that framework to the student question of how to prepare for the future. The central claim is that AI has shifted the interface of software work upward, but it has not removed the need for programming knowledge. Instead, it has increased the importance of verification, system-level reasoning, and accountability.

II. RELATED WORK

Recent literature shows that AI-assisted coding is no longer a side trend. The *Stack Overflow Developer Survey 2025* found that 84% of respondents were already using or planning to use AI tools in development, and about 51% of professional developers reported using them daily [1]. These numbers indicate that AI-assisted workflows have moved into the mainstream. The same survey also found that developers most commonly use these tools for writing code, searching for answers, learning new technologies, and debugging, which suggests that adoption is concentrated in drafting and support tasks rather than in full autonomy [1].

At the same time, related work shows that high usage does not imply high trust. Stack Overflow reported that more developers distrusted AI accuracy than trusted it, with 46% expressing distrust and 33% expressing trust; only about 3% said they highly trusted the output [1]. This is an important baseline for interpreting current methods. AI coding tools are being used widely, but they are not generally treated as independent authorities. Figure 1 summarizes this combination of mainstream use and limited confidence.

The literature on vibe coding provides a more detailed account of how these workflows operate. Brennan’s discussion

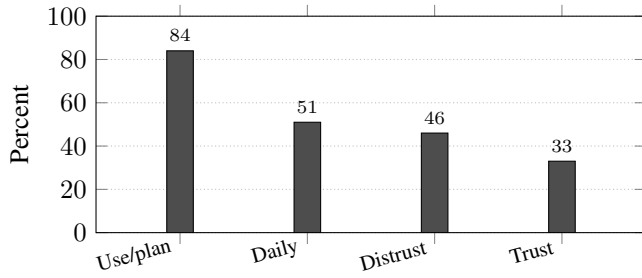


Fig. 1. Adoption and trust signals from Stack Overflow 2025 [1].

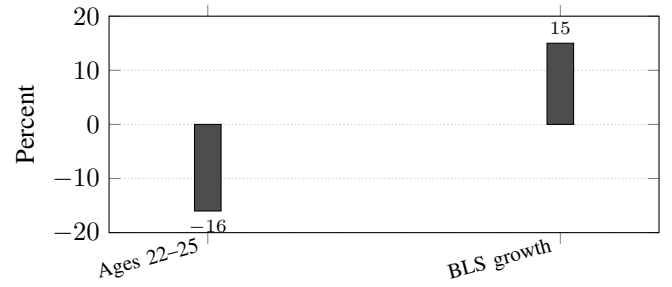


Fig. 2. Mixed labor-market signals.

in the *Harvard Gazette* defines vibe coding as creating software with AI assistance without necessarily understanding the generated code [2]. Her account is useful because it captures both the attraction and the limitation of the method. It makes prototyping easier, but students can become trapped in loops where the output is close to correct while the underlying problem remains unclear. Sarkar and Drosos provide the first empirical study of this workflow and show that vibe coding is not simply prompt-in, program-out. Instead, it consists of iterative cycles of prompting, scanning code, testing, manual editing, and deciding when to stop relying on the tool and intervene directly [3]. Their results suggest that programming expertise is not removed but redistributed toward context management, rapid evaluation, and selective control.

Another important branch of related work concerns performance and software quality. METR studied experienced open-source developers working in mature repositories and found that, in that setting, access to frontier AI tools actually made developers 19% slower on completed tasks, even though the developers expected speed gains [4]. This result does not imply that AI never helps. It does show that mature codebases, architectural constraints, review overhead, and debugging costs can erase the speed advantage of generation. Similarly, Cotroneo, Improta, and Liguori found that AI-generated code can exhibit distinct defect profiles and higher-risk vulnerabilities, even while remaining structurally simple and repetitive [5]. These studies reinforce the argument that generated code must still be inspected as engineering work, not merely accepted as output.

Finally, labor-market publications provide a mixed but useful context. The Budget Lab at Yale found no clear evidence of economy-wide labor disruption attributable to AI at the broad level [6]. By contrast, Brynjolfsson, Chandar, and Chen reported a 16% relative decline in employment for workers ages 22 to 25 in the most exposed occupations [7]. The U.S. Bureau of Labor Statistics still projects 15% growth from 2024 to 2034 for software developers, quality assurance analysts, and testers, with approximately 129,200 openings per year [8]. Taken together, these publications suggest not collapse, but reorganization, especially around the entry-level ramp. Figure 2 shows why the labor story is mixed rather than uniformly negative.

III. PROPOSED METHOD

The existing literature is useful but fragmented. Survey data describes adoption. Workflow studies describe how people interact with tools. Labor research describes hiring effects. Software-quality studies describe defects and vulnerabilities. What is still missing is a simple analytical structure that connects these findings to the student question. To address that gap, this paper proposes the Hybrid Responsibility Framework.

The Hybrid Responsibility Framework divides AI-assisted software work into three layers:

- 1) **Generation layer:** tasks where the main goal is to produce an initial artifact quickly. Typical examples include drafting code, boilerplate generation, documentation, API lookup, and early prototyping.
- 2) **Verification layer:** tasks where the output must be checked, tested, debugged, and compared against requirements. Typical examples include code review, tracing logic, writing tests, identifying edge cases, and correcting subtle failures.
- 3) **Accountability layer:** tasks where consequences are highest and human ownership is unavoidable. Typical examples include architecture, security, deployment, maintenance, long-term reliability, and coordination across teams and systems.

The value of this method is that it explains why public impressions of AI-assisted programming often feel contradictory. At the generation layer, AI can appear highly capable because speed and drafting matter most. At the verification layer, the same systems can become slower or less reliable because near-correct output still demands careful human checking. At the accountability layer, the human role remains dominant because someone must ultimately defend the result in production.

Figure 3 summarizes the framework.

This method is original to this paper as its primary analytical contribution. It does not claim to be a new algorithm. Instead, it is a new analytical model for organizing recent evidence and translating that evidence into educational guidance.

IV. METHOD IMPLEMENTATION

The Hybrid Responsibility Framework was implemented as a structured literature-synthesis procedure. Each selected source was mapped to one or more of the three layers based on the main phenomenon it measured.

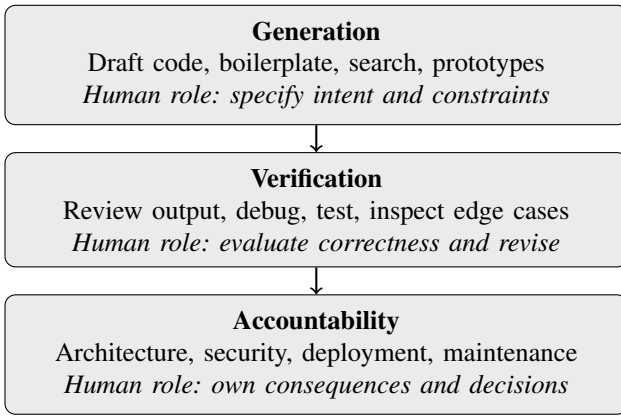


Fig. 3. Hybrid Responsibility Framework.

Survey evidence from Stack Overflow was mapped primarily to the generation and verification layers because it reports where developers use AI tools, how much they trust the outputs, and which tasks they avoid [1]. Workflow research on vibe coding was mapped across generation and verification because it describes how prompting, scanning, testing, and editing interact in practice [2], [3]. The METR study and the software-quality study were mapped mainly to verification because they measure what happens after code generation, especially in debugging, review, and correctness-sensitive settings [4], [5]. Labor-market studies were mapped to the accountability layer because they reflect how organizations value independent judgment, experience, and production responsibility over time [6], [7], [8].

This implementation also produced a practical student workflow derived from the framework:

- 1) define the problem in natural language,
- 2) generate a draft with clear constraints,
- 3) inspect the code line by line,
- 4) test the result against expected behavior,
- 5) revise weak parts manually,
- 6) and accept ownership for the final system.

In effect, the implementation translates the framework into a repeatable way of using AI assistance without treating it as an oracle. Although the paper does not present a software simulation, it does provide an operational model that students can apply directly in coursework and projects.

V. ANALYSIS RESULTS

Applying the framework to the evidence produces several clear results.

A. Result 1: AI Is Strongest at the Generation Layer

The adoption data strongly supports the conclusion that AI tools are most effective and most accepted at the generation layer. Developers use them heavily for writing code, searching for answers, and learning unfamiliar concepts [1]. This pattern fits the strengths of language models: they produce drafts quickly, summarize documentation well, and lower the cost

of experimentation. Vibe coding and rapid prototyping are therefore not illusions. They reflect a genuine gain in software leverage.

B. Result 2: Verification Remains the Main Bottleneck

The generation advantage does not eliminate the need for verification. Survey respondents most frequently complained that AI-generated solutions were “almost right, but not quite,” and many also reported that debugging AI-generated code took more time [1]. METR’s findings reinforce this pattern by showing that experienced developers can still be slowed down once review and correction are included [4]. The framework explains this clearly: speed at the generation layer does not automatically transfer to speed at the verification layer.

C. Result 3: Accountability-Heavy Tasks Still Require Programming Depth

The evidence shows that the human role becomes strongest where consequences become serious. Stack Overflow found strong resistance to using AI for deployment and monitoring, as well as for project planning [1]. This resistance is rational. These are not merely text-generation tasks. They involve tradeoffs, reliability, coordination, and responsibility. Security studies also support this conclusion because even plausible-looking code can contain defects and vulnerabilities that only careful human review will catch [5]. For students, this means that software engineering fundamentals are not obsolete. They are the skills that convert generated output into dependable systems.

D. Result 4: The Labor Market Is Shifting Upward Rather Than Disappearing

The labor-market evidence suggests a selective rather than universal effect. At the broad level, Yale’s findings argue against dramatic claims of immediate replacement across the whole economy [6]. At the same time, Stanford’s evidence suggests that younger workers in highly exposed jobs may already be facing pressure [7]. BLS projections remain positive for software employment overall [8]. The framework interprets these findings as a shift in employer expectations. If AI compresses some lower-level drafting work, then firms may demand more from entry-level hires, especially in independent problem solving and system judgment.

E. Discussion

These results lead to an important educational conclusion. Students should not stop learning to code. They should stop treating coding and prompting as opposites. Prompting is best understood as a specification skill. It helps produce drafts, narrow search space, and accelerate iteration. Programming knowledge is what makes that output usable. A student who can generate code but cannot explain it remains vulnerable at the verification and accountability layers. A student who understands software deeply but refuses modern tools may work accurately, but less efficiently than peers who can combine both modes.

Therefore, the strongest student preparation strategy is hybrid competence. That includes coding fundamentals such as data structures, debugging, testing, databases, APIs, and security, along with newer skills such as context-setting, writing precise prompts, asking for tests and explanations, and reviewing generated code critically. Figure 4 summarizes why the hybrid approach is stronger than either extreme on its own.

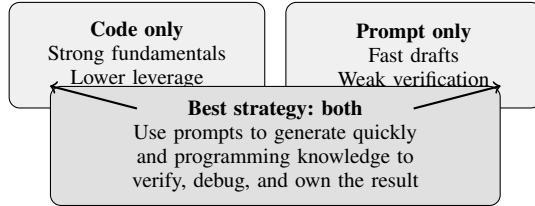


Fig. 4. Student preparation strategy: hybrid competence is strongest.

The future developer is less likely to be the person who types every line manually and more likely to be the person who can direct tools well while still knowing when the tools are wrong.

VI. CONCLUSION AND FUTURE WORK

This paper examined whether programming is being replaced by prompting and argued that the better interpretation is reorganization rather than replacement. After reviewing recent literature, it proposed the Hybrid Responsibility Framework as a new analytical method for separating AI-assisted software work into generation, verification, and accountability layers. Implementing this framework across survey, workflow, labor-market, and software-quality evidence showed that AI is already powerful for drafting and exploration, but much less decisive in correctness-sensitive and consequence-heavy work.

The main objective conclusion is that programming still matters because software work is not finished when code is generated. It is finished when the code is correct, secure, maintainable, and defensible. For students, this means the future is not coding alone and not prompting alone. It is learning how to use both intelligently.

Future work could extend this paper in a more empirical direction. One useful next step would be a classroom experiment comparing student performance under three conditions: manual coding only, AI-assisted generation without structured verification, and AI-assisted generation with the Hybrid Responsibility Framework. Another direction would be to evaluate whether portfolios built with explicit evidence of review, testing, and revision are viewed more favorably by employers than portfolios that present only polished outputs. Those studies would help test the framework more directly.

REFERENCES

- [1] Stack Overflow, “2025 Stack Overflow Developer Survey: AI,” 2025. [Online]. Available: <https://survey.stackoverflow.co/2025/ai/>
- [2] J. Sweet, “Vibe coding may offer insight into our AI future,” *Harvard Gazette*, Apr. 1, 2026. [Online]. Available: <https://news.harvard.edu/gazette/story/2026/04/vibe-coding-may-offer-insight-into-our-ai-future/>

- [3] A. Sarkar and I. Drosos, “Vibe coding: programming through conversation with artificial intelligence,” in *Proc. 36th Annual Conference of the Psychology of Programming Interest Group*, 2025. [Online]. Available: <https://arxiv.org/abs/2506.23253>
- [4] METR, “Measuring the impact of early-2025 AI on experienced open-source developer productivity,” 2025. [Online]. Available: https://metr.org/Early_2025_AI_Experienced_OS_Devs_Study-paper.pdf
- [5] D. Cotroneo, C. Improta, and P. Liguori, “Human-written vs. AI-generated code: a large-scale study of defects, vulnerabilities, and complexity,” in *Proc. IEEE International Symposium on Software Reliability Engineering*, 2025. [Online]. Available: <https://arxiv.org/abs/2508.21634>
- [6] M. Gimbel, M. Kinder, J. Kendall, and M. Lee, “Evaluating the impact of AI on the labor market: current state of affairs,” The Budget Lab at Yale, Oct. 1, 2025. [Online]. Available: <https://budgetlab.yale.edu/research/evaluating-impact-ai-labor-market-current-state-affairs>
- [7] E. Brynjolfsson, B. Chandar, and R. Chen, “Canaries in the coal mine? Six facts about the recent employment effects of artificial intelligence,” Stanford Digital Economy Lab, Working Paper, Nov. 13, 2025. [Online]. Available: <https://digitaleconomy.stanford.edu/publication/canaries-in-the-coal-mine-six-facts-about-the-recent-employment-effects-of-artificial-intelligence/>
- [8] U.S. Bureau of Labor Statistics, “Software developers, quality assurance analysts, and testers,” *Occupational Outlook Handbook*, Sep. 3, 2025. [Online]. Available: <https://www.bls.gov/ooh/Computer-and-Information-Technology/Software-developers.htm>